

PYTHON FILE HANDLING & AUTOMATION

Internship Project Report | Alphido Tech

1. Project Objective

The objective of this project is to implement core Python file-handling concepts and develop automated scripts to streamline basic file management operations efficiently.

2. Technologies & Modules Used

Language: Python 3.x

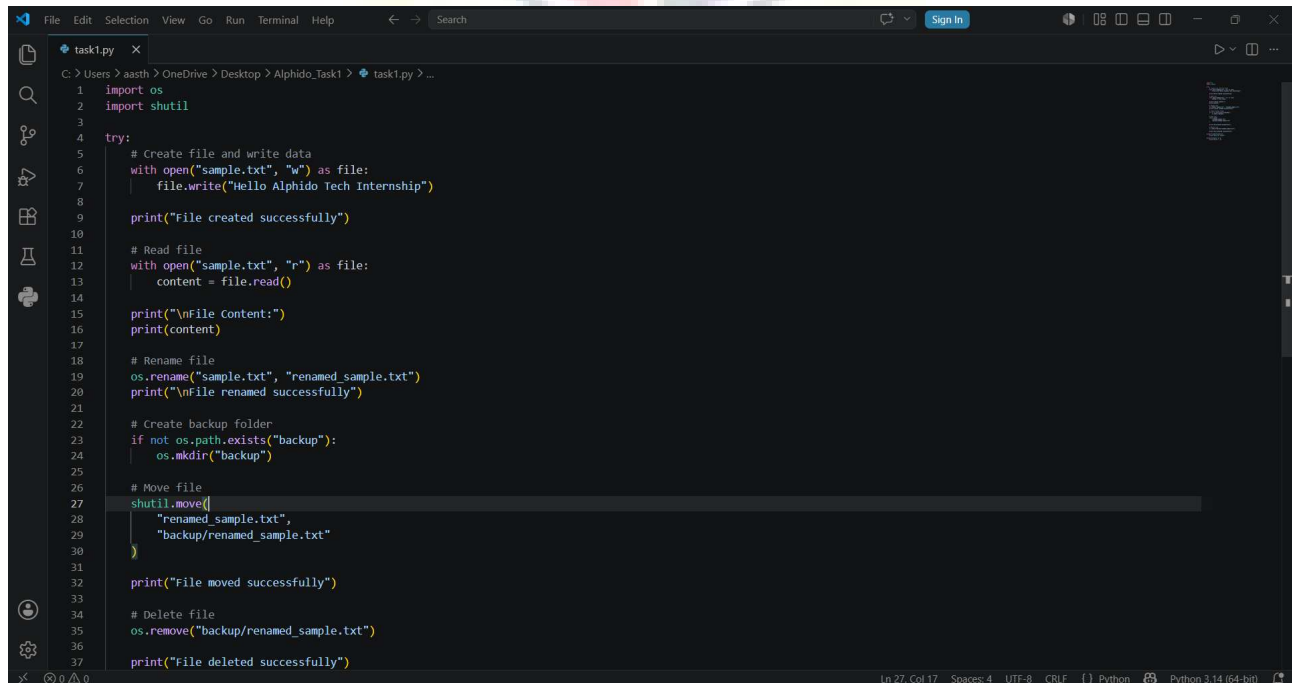
Development Environment: Visual Studio Code (VS Code)

Core Modules: * OS (Operating System interaction)

Shutil (High-level file operations)

3. Project Description

This project demonstrates the practical implementation of automated file management. The script execution flows through a sequence of programmatic operations protected by robust error-handling mechanisms.



```
1 import os
2 import shutil
3
4 try:
5     # Create file and write data
6     with open("sample.txt", "w") as file:
7         file.write("Hello Alphido Tech Internship")
8
9     print("File created successfully")
10
11    # Read file
12    with open("sample.txt", "r") as file:
13        content = file.read()
14
15    print("\nFile Content:")
16    print(content)
17
18    # Rename file
19    os.rename("sample.txt", "renamed sample.txt")
20    print("\nFile renamed successfully")
21
22    # Create backup folder
23    if not os.path.exists("backup"):
24        os.mkdir("backup")
25
26    # Move file
27    shutil.move(
28        "renamed sample.txt",
29        "backup/renamed_sample.txt"
30    )
31
32    print("File moved successfully")
33
34    # Delete file
35    os.remove("backup/renamed_sample.txt")
36
37    print("File deleted successfully")
```

The automated pipeline executes the following operational steps:

File Initialization: Programmatically creates a new text file.

Data Ingestion: Writes specified text data into the newly created file.

Data Verification: Reads and displays the contents of the file within the console.

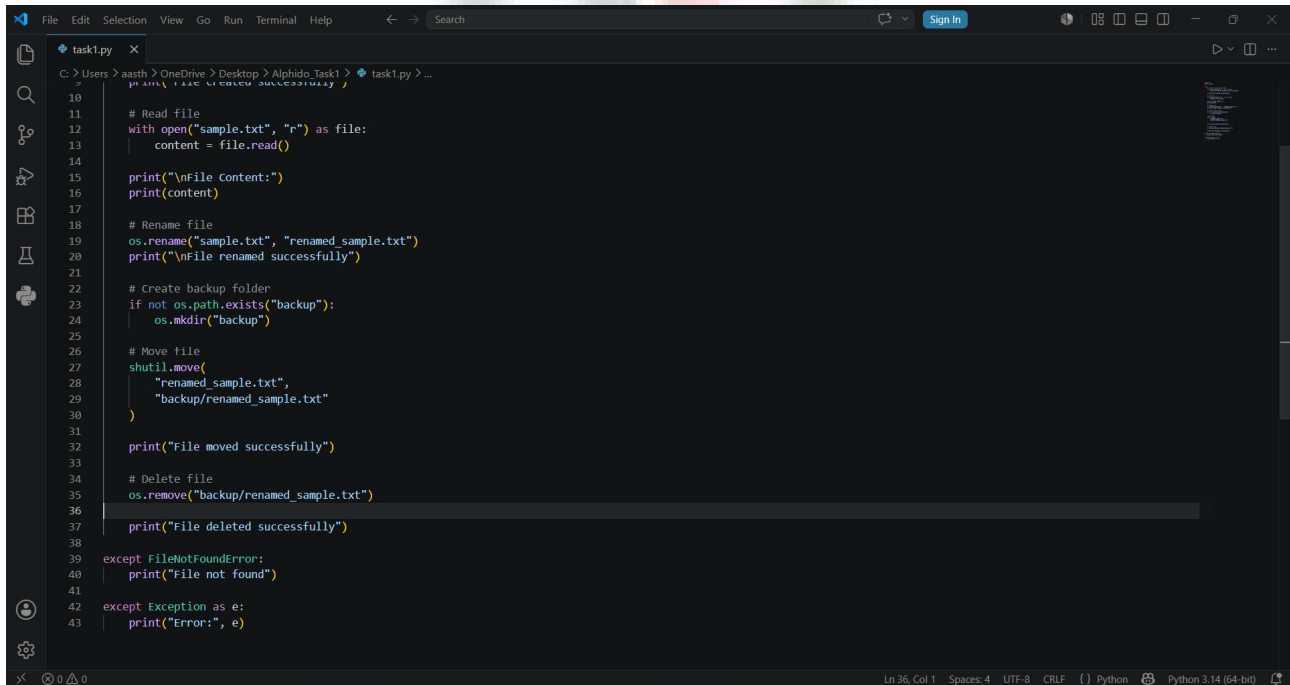
File Renaming: Safely renames the existing file to a specified target name.

Directory Management: Creates a dedicated backup directory if it does not already exist.

File Relocation: Moves the renamed file into the newly created backup directory.

Resource Cleanup: Deletes the file securely after verifying the operations.

Exception Handling: Wraps operations in try-except blocks to prevent system crashes.



```
10
11 # Read file
12 with open("sample.txt", "r") as file:
13     content = file.read()
14
15 print("\nFile Content:")
16 print(content)
17
18 # Rename file
19 os.rename("sample.txt", "renamed_sample.txt")
20 print("\nFile renamed successfully")
21
22 # Create backup folder
23 if not os.path.exists("backup"):
24     os.mkdir("backup")
25
26 # Move file
27 shutil.move(
28     "renamed_sample.txt",
29     "backup/renamed_sample.txt"
30 )
31
32 print("File moved successfully")
33
34 # Delete file
35 os.remove("backup/renamed_sample.txt")
36
37 print("File deleted successfully")
38
39 except FileNotFoundError:
40     print("File not found")
41
42 except Exception as e:
43     print("Error:", e)
```

4. Key Learning Outcomes

File I/O Proficiency: Developed hands-on skills in creating, reading, writing, renaming, moving, and deleting files using Python.

System Automation: Mastered the use of the os and shutil modules to interact directly with the operating system file system.

Defensive Programming: Gained practical experience implementing robust exception handling (try-except blocks) to manage filesystem errors (e.g., FileNotFoundError, PermissionError).

Problem Solving: Enhanced logical reasoning and debugging workflows for real-world backend automation.

5. Conclusion

This task provided valuable, hands-on experience in Python file handling and automation workflows. By bridging theoretical programming with practical scripts, it enhanced my core understanding of filesystem architecture, automated data management, and operational error-handling techniques essential for modern software development.

Prepared By:

Aastha Dhiman

Python Intern — Alphido Tech

BCA (AI & ML)

